



**BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONOURS**

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II: SESSION 2025/2026

PROJECT REPORT

Project Title:

Microservices-Based Disaster Relief Coordination Platform

GROUP 02

COURSE LECTURER:

DR. MOHAMAD NOR HASSAN

NAME	MATRIX NUMBER
MUHAMMAD ADHA BIN MUHAMMAD HAMKA	S76187
MUHAMMAD AIMAN MUKHLIS BIN KHAIRUM ANUAR	S76194
MUHAMMAD SYAKIR ZUFAYRI BIN SAARI	S74824

TABLE OF CONTENT

1. Introduction	3
2. Problem Description	4
3. System Requirements	5
4. Architecture Design	6
5. Architecture Diagrams	7
6. Technology Stack	8
7. Prototype Implementation	9
8. Architecture Evaluation	10
9. Team Contributions	11
10. Conclusion and Reflection	12

1. Introduction

Our project is to create a system for disaster relief coordination designed to replace traditional monolithic systems, which are often too rigid and centralized to handle the chaos of emergencies. In a monolithic architecture, all features are tightly coupled into a single, massive program, meaning that a small change or a failure in one part of the system may affect the entire platform, making maintenance time-consuming and risky. These traditional designs often become slow or even crash when too many people try to access them at once, causing delays in rescue operations during critical situations. To overcome these limitations, the platform utilizes a microservices architecture that breaks the system down into a suite of independent services to provide fast coordination and reduce the effects of disasters in Malaysia, such as floods, earthquakes, or pandemics.

To achieve the necessary national scalability and robust fault isolation, the platform is structured entirely around a microservices architecture style. By decomposing distinct business capabilities into modular, independent domains such as dedicated reporting processing, resource tracking, and volunteer assignment, high-traffic spikes targeting a single function can be managed precisely without threatening overall system uptime. The platform caters to a diverse ecosystem of end users with highly specific operational roles. Victims can rapidly broadcast highly accurate GPS coordinates paired with localized photo evidence. Simultaneously, Volunteers register their specialized skillsets and dynamically accept targeted rescue tasks. On the strategic coordination front, Non-Governmental Organizations (NGOs) and Government Agencies access the platform to continuously monitor real-time stock inventories, dispatch critical relief supplies within strict geographic boundaries, and audit official communications.

2. Problem Description

Through analyzing the problem, I realized that disaster management systems play a critical role in ensuring that rescue operations and relief efforts are carried out efficiently. During emergencies, a large number of users may simultaneously report incidents, request assistance, or access important information. If the system cannot handle this high demand, delays may occur and negatively impact affected communities. This highlighted the importance of scalability and system reliability in disaster management applications.

I also learned that effective communication and coordination between stakeholders are essential for successful disaster response. Government agencies, NGOs, volunteers, and relief centers need access to the same up-to-date information to make informed decisions. Without real-time information sharing, resources may be distributed inefficiently, rescue efforts may overlap, and some affected areas may not receive the assistance they urgently require.

Another important insight is the limitation of traditional centralized systems. Since their components are tightly connected, maintaining and upgrading the system becomes more challenging as it grows. Even a small modification can affect other parts of the system, increasing development time and the risk of unexpected errors. This can make it difficult for organizations to adapt the system to changing disaster response requirements.

Based on these observations, I believe that a microservices architecture is a suitable solution for addressing these challenges. By dividing the system into independent services, each component can be developed, deployed, and maintained separately without affecting the entire platform. This approach improves scalability, flexibility, and fault tolerance while supporting real-time communication and coordination among stakeholders. As a result, the proposed system can provide a more effective and reliable platform for disaster relief management.

3. System Requirements

Functional Requirements

1. User & Authentication Module

- a. FR-AUTH-01: The system shall validate user registration inputs (Victims, NGOs, Agencies, Volunteers) ensuring unique email addresses and password
- b. FR-AUTH-02: The API Gateway shall intercept all Incoming service requests and validate a JWT (JSON Web Token) before granting access to downstream microservices.

2. Reporting & Prioritization Module

- a. FR-REP-01: The system shall capture and store GPS coordinates (latitude/longitude) with high accuracy.
- b. FR-REP-02: The system shall automatically assign a "High" priority status to reports containing keywords such as "Trapped," "Medical Emergency," or "Flood Rising."
- c. FR-REP-03: The reporting module shall limit photo uploads to a maximum of 3 files per report, with a size limit of 5MB per image.

3. Resource Management Module

- a. FR-RSRC-01: Upon a resource request, the system shall query the inventory database and return a list of relief centers within a 50km radius that have the requested items in stock.
- b. FR-RSRC-02: The system shall decrement the "Available Stock" count in the database automatically once a delivery is marked as "Completed" by the driver.

4. Volunteer Management Module

- a. FR-VOL-01: System shall allow volunteers to register skills (medical, rescue, etc.) and automatically matches them to incidents based on location and skillset
- b. FR-VOL-02: Enables volunteers to accept or reject assigned tasks in real-time.

5. Communication Module

- a. FR-COM-01: Provides real-time messaging between victims, responders, NGOs, and government agencies.
- b. FR-COM-02 : The system shall log all communication history and message details to support official monitoring and post-disaster auditing

6. Notification Module

- a. FR-NOTIF-01: The system shall trigger alerts across multiple channels, including mobile push notifications (for app users), SMS, and email (for registered contacts)
- b. FR-NOTIF-02 : The system shall automatically trigger high-priority alerts for critical events, such as immediate evacuation orders, resource shortages, or high-risk incidents

Non-Functional Requirements

- Scalability and Performance
 - NFR-PERF-01: The system must handle sudden surges in user traffic without causing the platform to slow down or crash.
 - NFR-PERF-02: The microservices architecture must allow for the precise allocation of server resources specifically to high-demand services (such as incident reporting) during a disaster, without requiring a full-system infrastructure upgrade.
 - NFR-PERF-03: The decentralized data architecture must efficiently manage high volumes of concurrent read and write operations without creating data bottlenecks.
- Reliability and Availability
 - NFR-REL-02: The system must exhibit strong fault isolation; the failure or overwhelming of a single service must not cause the entire application to collapse, ensuring other services remain fully active.
- Security
 - NFR-SEC-01: The API Gateway must intercept all incoming requests and validate a JSON Web Token (JWT) before granting access to any downstream microservices.
 - NFR-SEC-02: The system must securely handle and verify user credentials to guarantee that all registrations are distinct and secure.
- Maintainability and Deployment
 - NFR-MNT-01: Each microservice must be designed to focus on a distinct business task to allow for independent development, updates, and deployment without jeopardizing the stability of the entire platform.

4. Architecture Design

The Microservices-Based Disaster Relief Coordination Platform is proposed as a replacement for traditional, rigid monolithic systems, leveraging a microservices architecture. Centralized monolithic systems are highly susceptible to bottlenecks and potential crashes during disaster scenarios, especially when overwhelmed by a high volume of simultaneous user access. To mitigate these risks, the microservices approach is employed, which involves decomposing the application into a collection of independent and modular services.

- **High Scalability and Cost-Efficiency:**

- In emergencies, the surge in traffic typically targets specific functions, such as incident reporting. This microservices architecture enables precise allocation of server resources to the services experiencing heavy demand, avoiding the need for a full-system infrastructure upgrade.

- **Fault Isolation and Resilience:**

- Should a single service fail, whether from an error or overwhelming demand, the overall application will continue to operate. Given that system uptime is critical for a disaster relief platform and directly impacts the ability to save lives, this fault tolerance is essential.

- **Decentralized Data Management:**

- Each service operates with its own dedicated data store, eliminating reliance on a single central database. This architectural approach efficiently manages high volumes of concurrent reads and writes and avoids data bottlenecks across different modules.

- **Independent Deployment:**

- Focusing on a distinct business task, each microservice allows for independent development, updates, and deployment, which ensures the stability of the entire platform is not jeopardized.

5. Architecture Diagrams

System Context Diagram

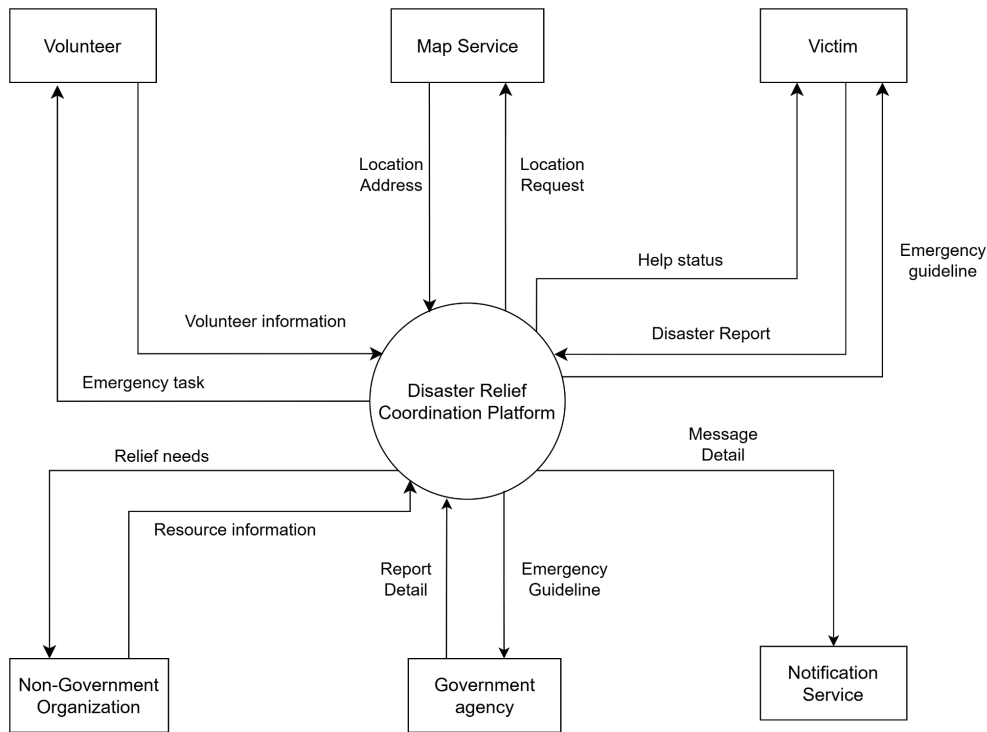


Figure 1.0 System Context Diagram

Description :-

1. Victim

- Input to Platform: Sends a Disaster Report to request help.
- Output from Platform: Receives Help status updates and Emergency guidelines for immediate safety instructions.

2. Volunteer

- Input to Platform: Provides Volunteer information (availability, skills, location).
- Output from Platform: Receives an Emergency task assigned by the system.

3. Government Agency (Official Government organization exp: Police,Ambulance and firefighter)

- Input to Platform: Provides Report Detail (official assessments or situational data).
- Output from Platform: Receives Emergency Guidelines for dissemination or implementation.

4. Non-Government Organization (NGO) Charity or private group

- Input to Platform: Provides Resource information (supplies, medical kits, food).
- Output from Platform: Receives Relief needs data to know where to deploy their resources.

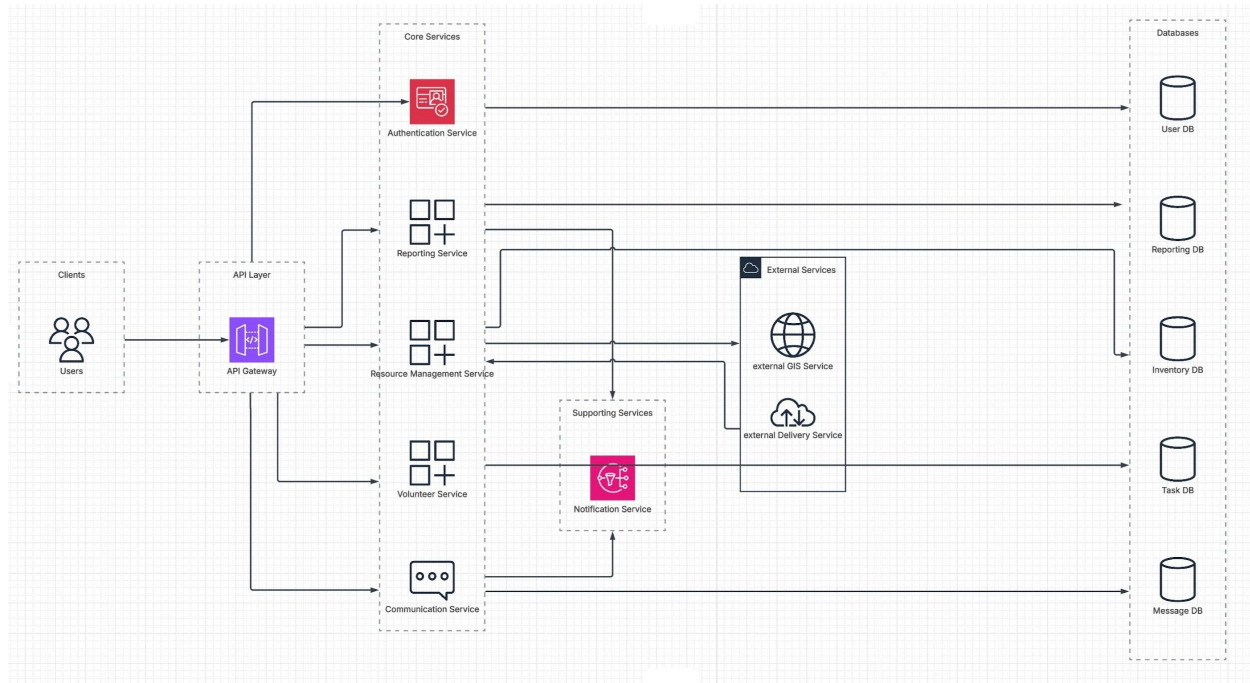
5. GIS Service AKA Map service

- Input to Platform: Sends Location Address data.
- Output from Platform: Receives a Location Request to geocode or map a specific report.

6. Notification Service (Communication like SMS and Push notification)

- Input to Platform: Sends Delivery status (confirming if a message reached its destination).
- Output from Platform: Receives Message Detail to be broadcasted to users.

Service Architecture Diagram



1. Users start by going through the API Gateway.
2. The API Gateway sends requests to five services: Authentication, Reporting, Resource Management, Volunteer, and Communication.
3. Each service has its own database.
4. The Authentication Service handles login and user access, and it uses the User DB.
5. The Reporting Service creates reports, stores data in the Reporting DB, and sends alerts through the Notification Service.
6. The Resource Management Service tracks supplies and resources, stores data in the Inventory DB, and connects to the GIS Service.
7. The Volunteer Service manages volunteer tasks and uses the Task DB.
8. The Communication Service handles messages, uses the Message DB, and also sends alerts through the Notification Service.
9. The Notification Service is shared by the Reporting and Communication services.
10. The Delivery Service connects to the Resource Management Service to help with delivery and logistics.

11. Overall, the diagram shows a system where each part has a clear job and works together to support disaster relief.

Interaction Diagram

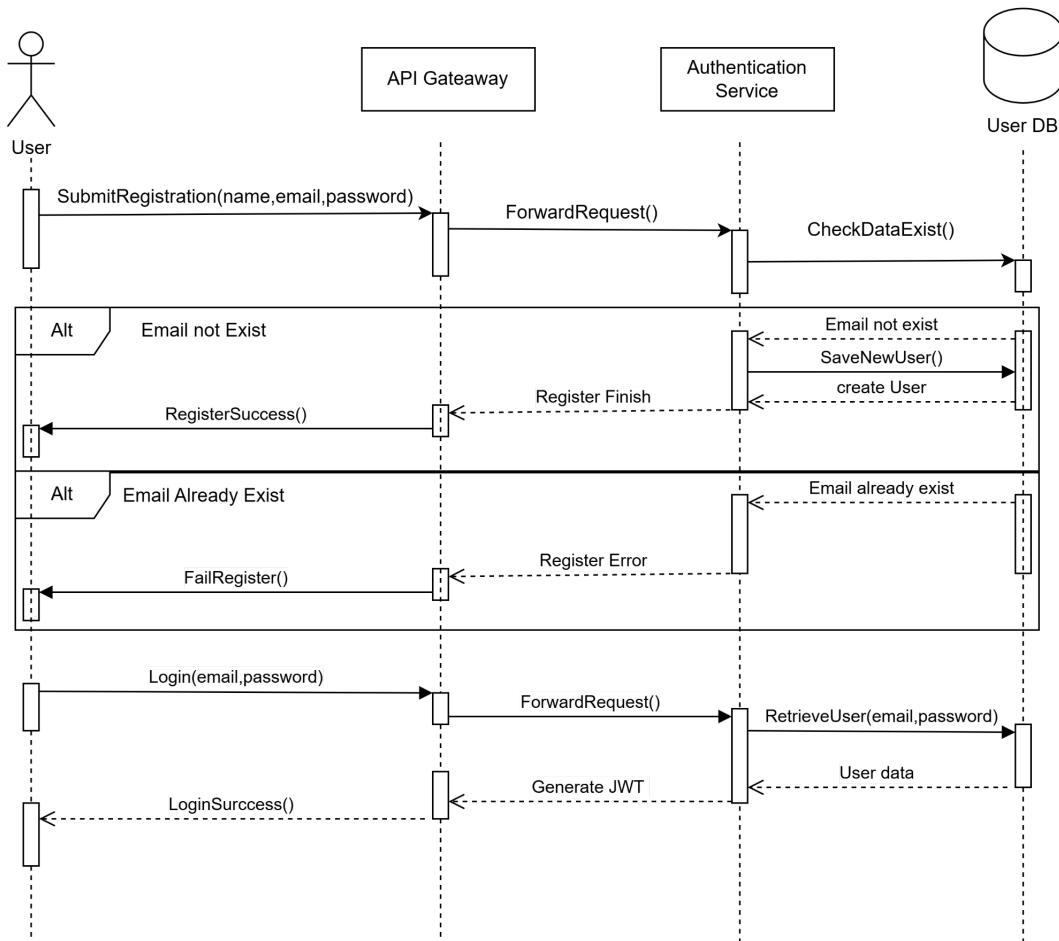


Figure Sequence Diagram Module User Registration & Login

Flow :-

1. The User submits their registration details, including their name, email, and password, to the API Gateway.
2. The API Gateway forwards this registration request to the Authentication Service for validation.
3. The Authentication Service checks with the User DB to see if the provided email address already exists in the system.
4. If the email does not exist:

- a. The Authentication Service instructs the User DB to save the new user's information.
 - b. Once the user is created, the Authentication Service notifies the API Gateway that the registration is finished.
 - c. The API Gateway confirms a successful registration to the User.
5. If the email already exists:
 - a. The User DB alerts the service that the email is a duplicate.
 - b. The Authentication Service sends a registration error message back to the API Gateway.
 - c. The API Gateway notifies the User that the registration has failed.
6. To access the system, the User submits their login credentials (email and password) to the API Gateway.
7. The API Gateway forwards the login request to the Authentication Service.
8. The Authentication Service attempts to retrieve the matching user record from the User DB.
9. Upon receiving the valid user data from the database, the Authentication Service generates a JWT (JSON Web Token).
10. The API Gateway receives the token and informs the User that the login was successful.

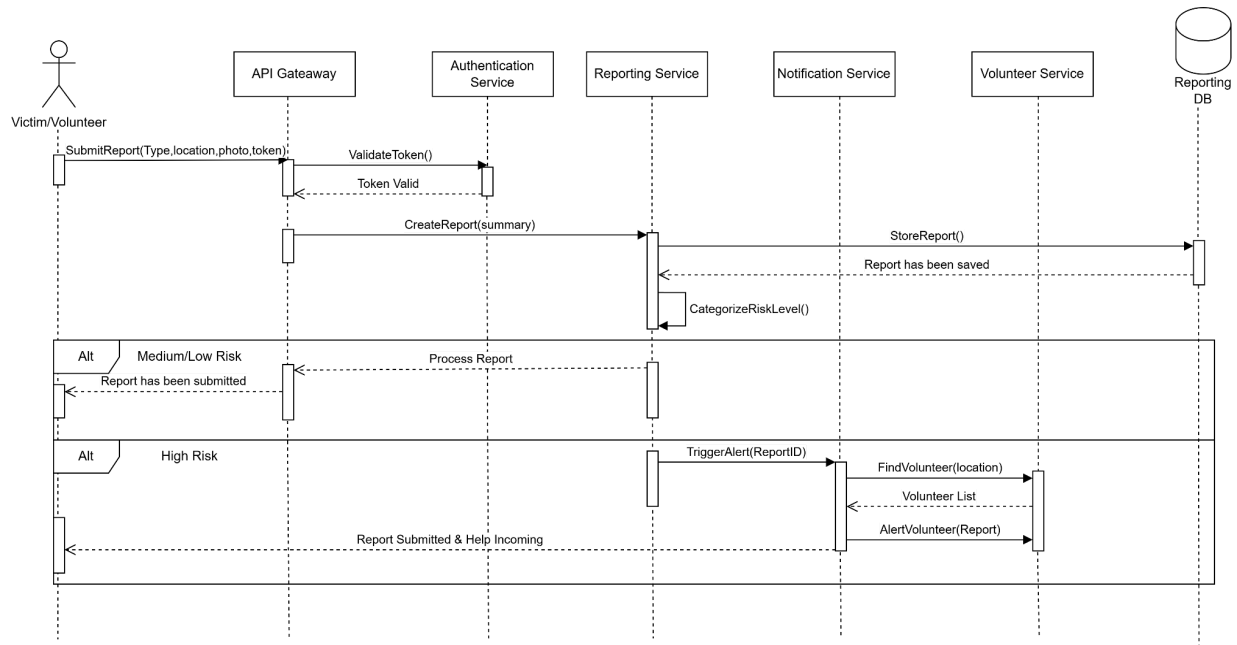


Figure Sequence Diagram Reporting Module

Flow :-

1. A Victim/Volunteer submits a report (type, location, photo, token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service to validate it.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway forwards the report request to the Reporting Service.
5. The Reporting Service creates the report summary.
6. The Reporting Service stores the report in the Reporting Database.
7. The database confirms that the report has been saved.
8. The Reporting Service categorizes the risk level of the report.
 - a. If the report is Medium or Low Risk, the system processes the report and sends a confirmation that the report has been submitted.
 - b. If the report is High Risk, the Reporting Service triggers an alert in the Notification Service.
 - i. The Notification Service requests nearby volunteers from the Volunteer Service based on the report location.
 - ii. The Volunteer Service returns a list of available volunteers.
 - iii. The Notification Service sends alerts to the selected volunteers.

- iv. The system informs the user that the report has been submitted and help is on the way.

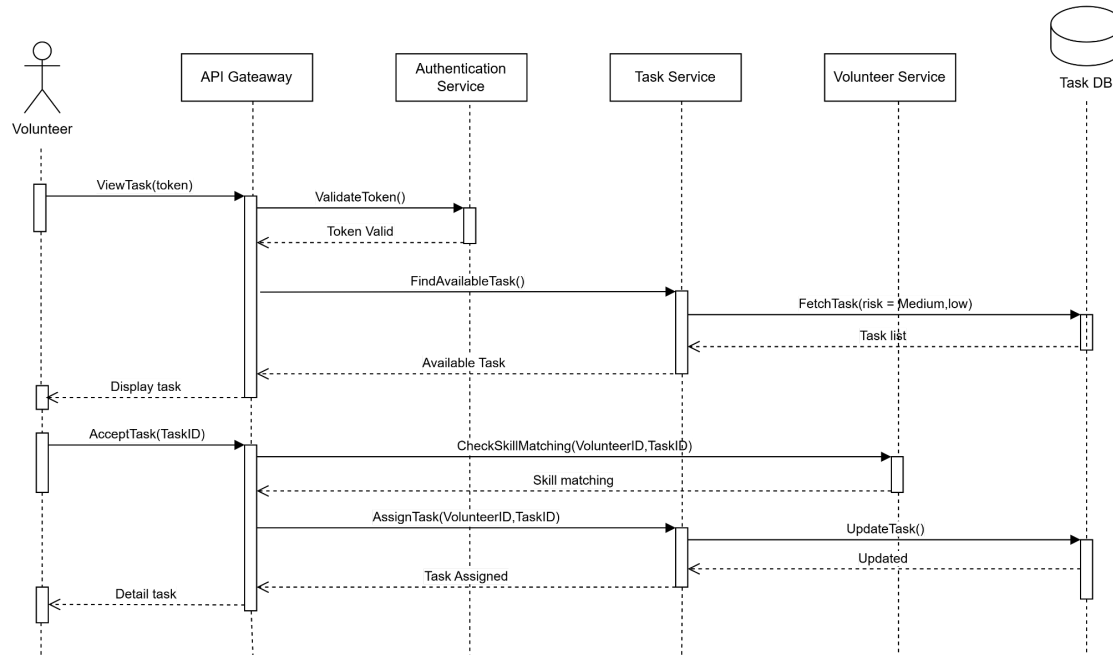


Figure Sequence Diagram Volunteer Management Module

Flow:-

1. A Volunteer sends a request to view tasks (with token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service for validation.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway requests available tasks from the Task Service.
5. The Task Service retrieves tasks from the Task Database with risk level (medium and low).
6. The Task Database returns the task list to the Task Service.
7. The Task Service sends the available tasks back to the API Gateway.
8. The API Gateway displays the task list to the Volunteer.
9. The Volunteer selects and sends a request to accept a task (TaskID) to the API Gateway.
10. The API Gateway requests the Volunteer Service to check skill matching using VolunteerID and TaskID.

11. The Volunteer Service verifies the skill compatibility and returns a “Skill matching” result.
12. The API Gateway sends a request to the Task Service to assign the task to the volunteer.
13. The Task Service updates the task in the Task Database.
14. The Task Database confirms that the task has been updated.
15. The Task Service returns a “Task Assigned” response to the API Gateway.
16. The API Gateway displays the task details to the Volunteer

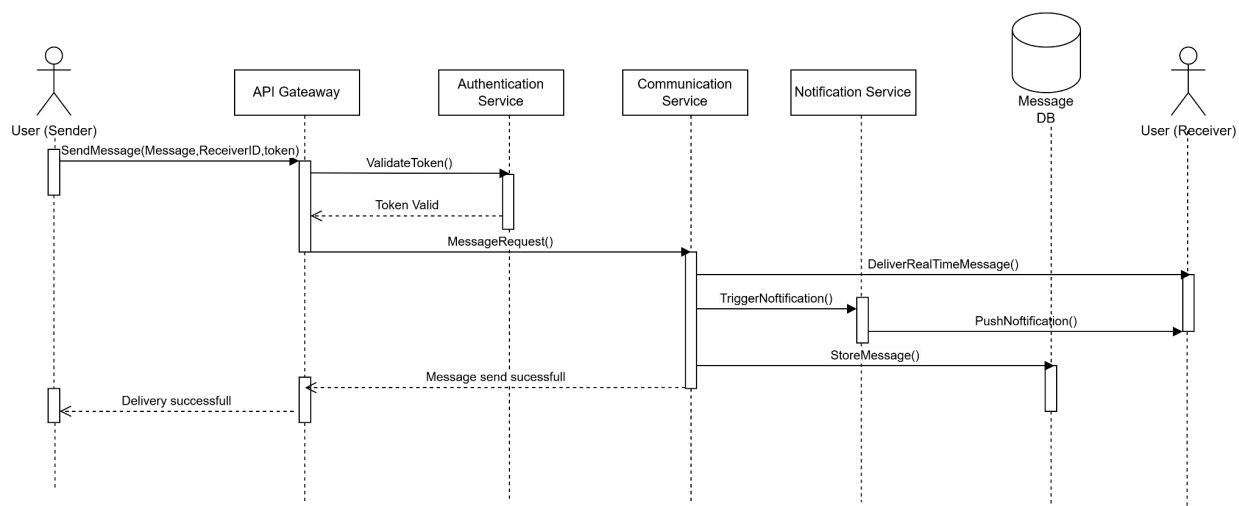


Figure Sequence Diagram Communication Module

Flow:-

1. A User (Sender) sends a message (message content, receiver ID, token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service for validation.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway forwards the message request to the Communication Service.
5. The Communication Service processes the message request.
6. The Communication Service delivers the message in real time to the User (Receiver).
7. The Communication Service triggers the Notification Service to notify the receiver.
8. The Notification Service sends a push notification to the User (Receiver).
9. The Communication Service stores the message in the Message Database.

10. The Communication Service returns a “Message send successful” response to the API Gateway.
11. The API Gateway sends a “Delivery successful” confirmation back to the User (Sender).

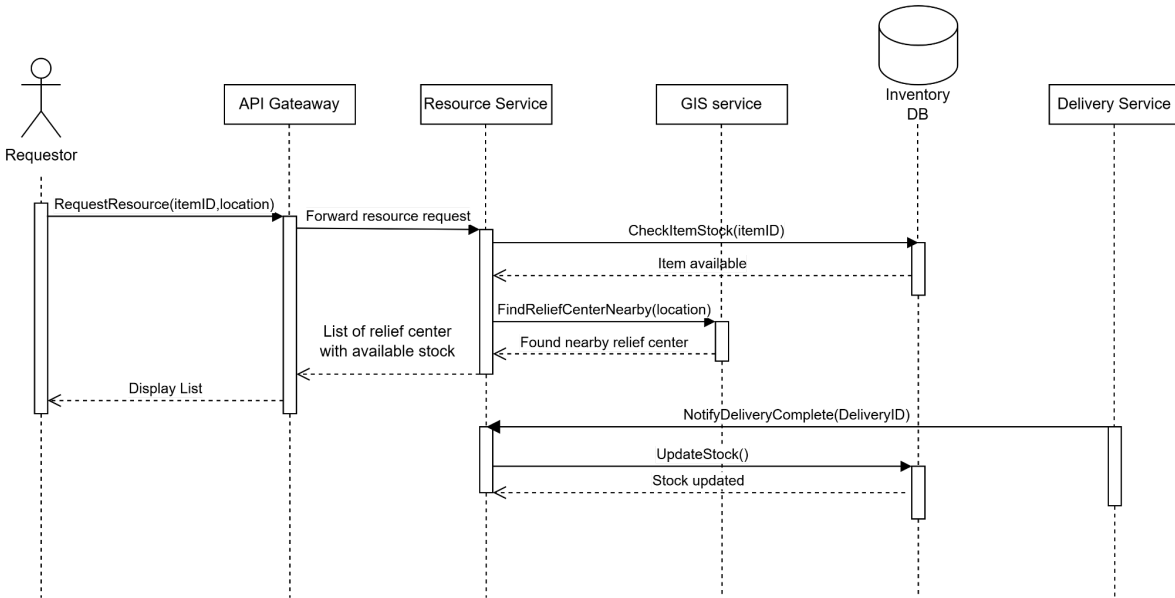


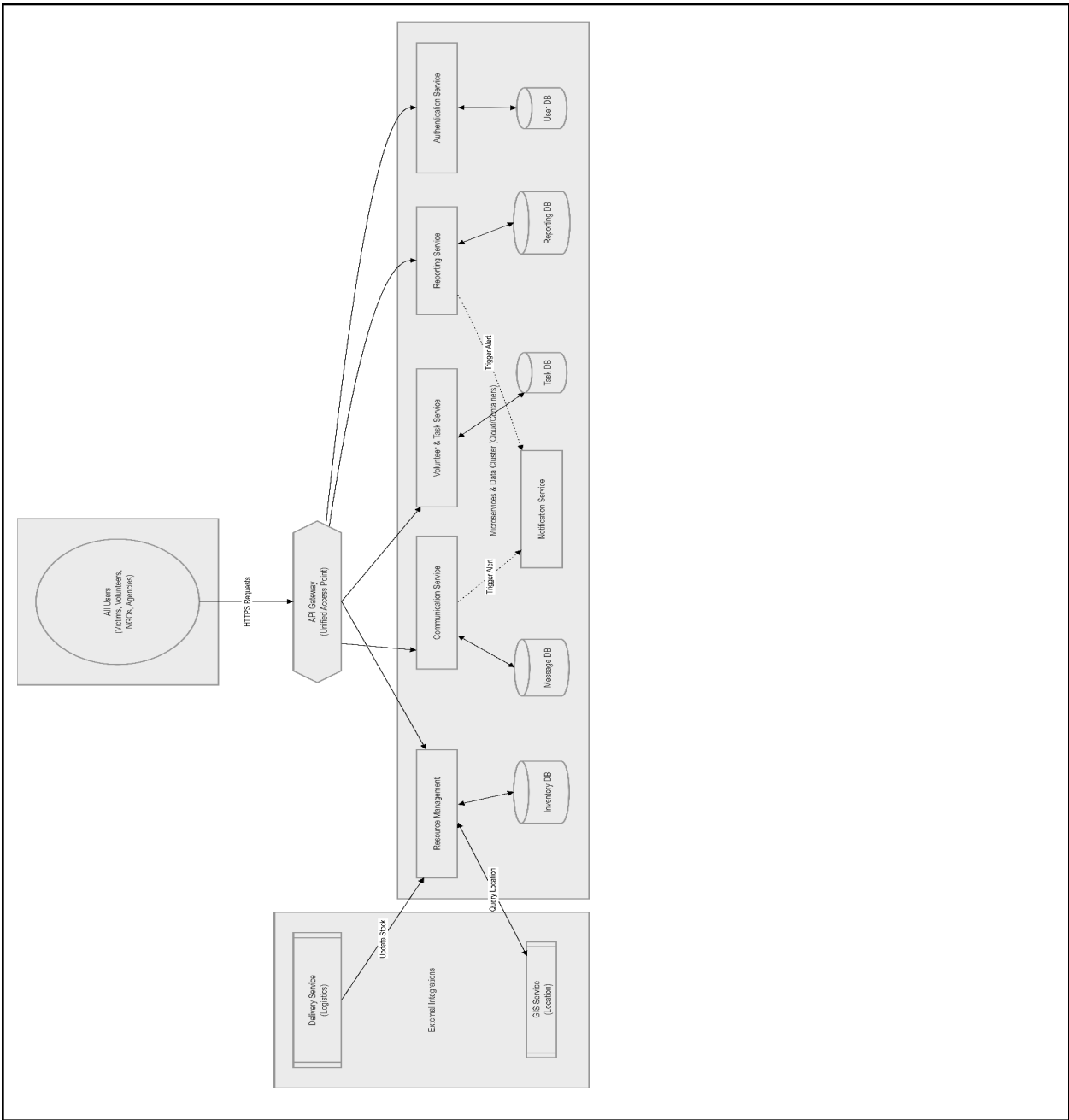
Figure Sequence Diagram Resource Management Module

Flow:-

1. The Requestor initiates a request for a specific resource by providing an item ID and their current location to the API Gateway.
2. The API Gateway acts as the entry point and forwards this request to the Resource Service for processing.
3. The Resource Service first queries the Inventory DB to verify if the requested item is currently in stock.
4. Once the Inventory DB confirms the item is available, the Resource Service contacts the GIS service to identify relief centers near the user's location.
5. The GIS service provides a list of nearby centers, which the Resource Service then filters based on where the stock is actually located.
6. This final list of available relief centers is sent back through the API Gateway and displayed to the Requestor.

7. At a later point, the Delivery Service triggers a notification to the Resource Service once a delivery has been successfully completed.
8. Upon receiving this completion notice, the Resource Service instructs the Inventory DB to update the stock levels to reflect the new delivery.
9. The Inventory DB confirms to the service that the records have been successfully updated.

Deployment Diagram



6. Technology Stack

- **Frontend User Interface**

- HTML5, CSS3 and Bootstrap 5 were utilized to create a responsive and accessible dashboard. Vanilla JavaScript was used for asynchronous client-side operations (To Fetch API) to communicate with the backend.

- **Backend Server (API)**

- [node.js](#) running the [express.js](#) framework was selected for the microservice. It provides a lightweight, fast environment ideal for handling asynchronous I/O operations and RESTful API routing.

- **Database Integration**

- MongoDB Atlas, a cloud-based NoSQL database was implemented. The mongoose Object Data Modelling (ODM) library was used to define structured schemas for the unstructured database.

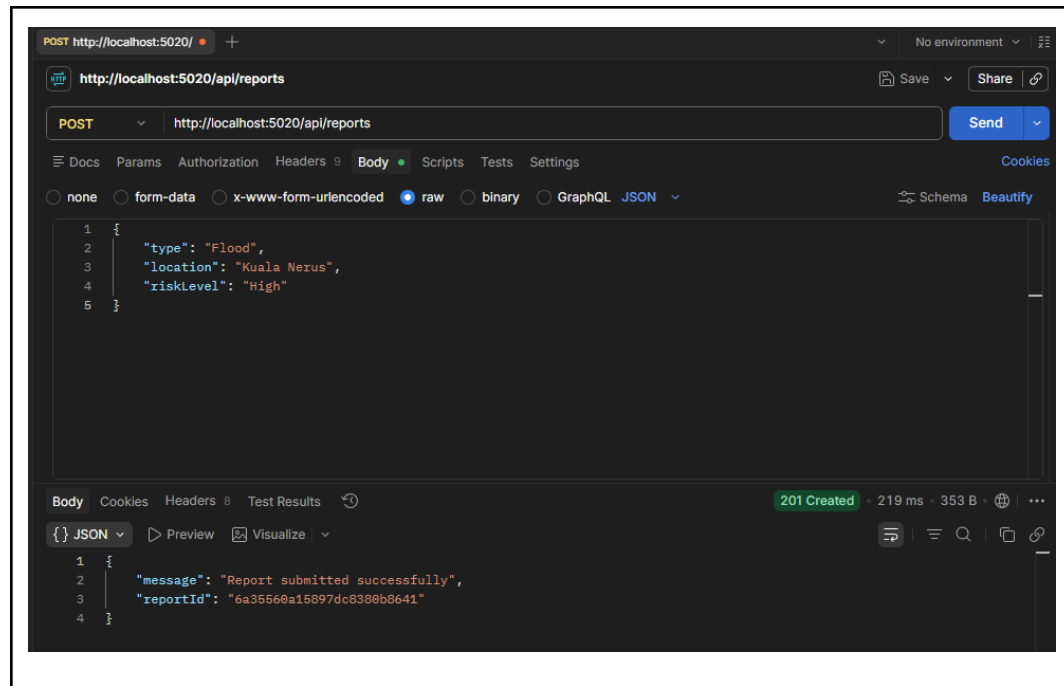
- **Middleware & Security**

- Cors package was implemented to securely manage Cross-Origin Resource Sharing, ensuring the API Gateway or Frontend can safely communicate with the isolated microservice port.

7. Prototype Implementation

7.1. Service Isolation & Communication

The Reporting microservice was deployed independently on **Port 5020**. The frontend UI acts as the client, sending standard HTTP requests (POST, GET, PATCH) to the service. A significant architectural achievement was configuring CORS middleware to successfully allow the isolated frontend to communicate with the backend server without triggering browser security blocks.



7.2. Cloud Database Migration

Initial rapid prototyping was conducted using a local SQLite database (reports.db). Once the data schema and API routes were verified, the architecture was successfully migrated to the cloud using MongoDB Atlas. This satisfied the requirement for a decentralized data architecture capable of handling high-volume disaster data.

7.3. Core Logic & Features

- **Data Ingestion:** A POST /api/reports endpoint securely receives JSON payloads from the frontend and stores them as documents in the MongoDB cluster.
- **Automated Prioritization Engine:** To satisfy FR-REP-02, the GET route implements an algorithmic sorting mechanism. It assigns weight to incident severity (High = 3, Medium = 2, Low = 1) and automatically returns the data feed sorted by urgency, ensuring critical emergencies are prioritized on the coordinator's dashboard regardless of submission time.
- **Status Management:** A PATCH endpoint allows coordinators to dynamically update the operational status of a report (Pending, In Progress, Resolved) directly from the dashboard, simulating real-time disaster coordination.

```
_id: ObjectId('6a353ac944350e189c32d687')
type: "Medical Emergency"
location: "Kuala Terengganu"
riskLevel: "High"
status: "Pending"
createdAt: 2026-06-19T12:49:13.734+00:00
__v: 0
```

```
_id: ObjectId('6a35560a15897dc8380b8641')
type: "Flood"
location: "Kuala Nerus"
riskLevel: "High"
status: "Pending"
createdAt: 2026-06-19T14:45:30.208+00:00
__v: 0
```

8. Architecture Evaluation

To ensure our Microservices-Based Disaster Relief Coordination Platform can withstand emergency traffic surges and isolated failures, we utilize the Architecture Tradeoff Analysis Method (ATAM). Rather than testing basic functionality, ATAM evaluates our system's underlying structure. By analyzing real-world stress scenarios, it uncovers risks and validates that our decentralized design successfully achieves critical non-functional requirements like scalability, maintainability, and reliable fault-tolerance.

Architecture Tradeoff Analysis Method (ATAM) :-

Scenario 1: Massive Flood Traffic Surge

- **Architectural Decision:** The architecture utilizes decentralized data management (MongoDB Atlas collections per service) and independent service scaling.
- **Analysis:** During the flood, the system automatically allocates maximum CPU and memory resources only to the Reporting Service and the API Gateway. The Resource and Volunteer services remain at baseline resources, saving costs. Because the Reporting DB is decoupled from the Inventory DB, high-volume write operations (saving reports) do not block read operations (checking resource stock).
- **Sensitivity Point:** The API Gateway's routing capacity and the Reporting database's write throughput are highly sensitive to this load.
- **Tradeoff (Scalability vs. Complexity):** We achieve high performance and targeted resource allocation, but at the tradeoff of increased operational complexity. Managing distributed databases is significantly harder than managing a single monolithic SQL database.

Scenario 2: Urgent Volunteer Module Update

- **Architectural Decision:** Microservices built around distinct business domains with independent deployment pipelines.
- **Analysis:** Developers update the Node.js code and MongoDB schema specifically for the Volunteer Service to include "Medical Certification." They deploy this update to

production independently. The Authentication, Reporting, and Resource services are never taken offline during this update.

- **Sensitivity Point:** The strictness of the bounded contexts. If the Volunteer database was tightly coupled to the Authentication database, this update would be much harder.
- **Tradeoff (Maintainability vs. Integration Testing):** The system achieves rapid, zero-downtime deployments for specific features. However, the tradeoff is that developers must conduct more rigorous integration testing to ensure the updated Volunteer Service still communicates correctly with the unchanged API Gateway and Task DB.

Scenario 3 : Resource Management Service Crashes

- **Architectural Decision:** Microservices operate with strict fault isolation and independent data stores, utilizing an API Gateway to route requests.
- **Analysis:** Due to an unexpected timeout from the external GIS map service, the Resource Management Service completely crashes. While victims and NGOs temporarily cannot search for nearby relief supplies or update inventory levels, the API Gateway continues routing critical traffic to the Reporting Service and Volunteer Service. Because the services are decoupled, victims can still successfully submit emergency reports, and volunteers can still accept rescue tasks without disruption.
- **Sensitivity Point:** The API Gateway's timeout and error-handling configuration (e.g., using a "circuit breaker" pattern). The Gateway must quickly recognize that the Resource Service is down and return a clean error to the user interface, rather than hanging indefinitely and consuming network connections waiting for a response.
- **Tradeoff (Core Availability vs. Feature Completeness):** The architecture trades the temporary loss of a specific feature (resource tracking) to guarantee the survival of the platform's most critical, life-saving functions (emergency reporting and volunteer dispatch). If this system were a monolith, a severe crash in the resource mapping module could have frozen the entire application for all users.

Scenario 4: Unauthorized Access Attack

- **Architectural Decision:** Centralizing all external communication through an API Gateway, which acts as the strict enforcement point for JWT (JSON Web Token) validation, while isolating user credential management entirely within a dedicated Authentication Service.
- **Analysis:** During a disaster, a malicious actor attempts to disrupt operations by bypassing the frontend dashboard and sending thousands of fake POST requests directly to the system's backend, hoping to create fake volunteer assignments or alter inventory data. However, because the microservices are not exposed to the public internet, the attacker must go through the API Gateway. The Gateway intercepts every request and searches for a valid JWT. Because the attacker lacks a JWT securely generated by the Authentication Service, the API Gateway immediately drops the requests with a 401 Unauthorized response. The downstream microservices and their databases are completely shielded from the malicious traffic.
- **Sensitivity Point:** The architecture is highly sensitive to the configuration of the API Gateway and the secrecy of the JWT signing key. If a developer accidentally leaves a microservice port (e.g., port 3003 for Resource Management) directly exposed to the public internet, bypassing the CORS and Gateway rules, the entire security model is compromised.
- **Tradeoff (Security vs. Latency and Bottlenecking):**
 1. **Latency:** Centralizing authentication provides robust, system-wide security, but it introduces a slight processing delay (latency) to every single request, as the Gateway must decode and verify the token signature before routing.
 2. **Single Point of Failure:** While the microservices themselves are fault-tolerant, making the API Gateway the mandatory security checkpoint creates a single point of failure. If the API Gateway is compromised or crashes under a DDoS attack, legitimate users cannot access any part of the system, even if the backend services are perfectly healthy.

9. Team Contributions

Team Member	Contribution
MUHAMMAD AIMAN MUKHLIS BIN KHAIROM ANUAR	• Defined the System Requirements (Functional & Non-Functional). • Designed the High-Level Architecture Diagrams, including the System Context Diagram and Interaction Diagram • Resource Management Module and Notification Module • Create Github repository to push for independent microservice • Architected the API Gateway
MUHAMMAD ADHA BIN MUHAMMAD HAMKA	• Conducted architecture research and analysis for the microservices approach. • Designed report prioritization logic and disaster response workflow. • Assisted in system integration and documentation review. • Validated system requirements against architectural decisions.
MUHAMMAD SYAKIR ZUFAYRI BIN SAARI	• Developer for the Reporting & Prioritization Module prototype. • Design the Deployment Diagram. • Implement Prototype into the system. • Divide the system module and assign it to other members.

10. Conclusion and Reflection

Designing the Microservices-Based Disaster Relief Coordination Platform was a very useful experience in the application of software architecture in solving practical problems. In this project, we managed to design a reliable and scalable platform that can help in organizing disaster relief activities through linking victims, volunteers, non-governmental organizations, and other government organizations into one single system.

Several benefits were observed with the implementation of the microservices approach as compared to monolithic systems. The independence of each service made it more scalable, fault tolerant, maintainable, and flexible for deployment. Due to the separation of the essential services of authentication, reporting, communication, resource management, and volunteer coordination, the system could keep on functioning even if there was a failure of individual services.

From a personal standpoint, I acquired practical experience in design of deployment architecture, microservice implementation and development of the report function to help with the operations of disaster management. The Reporting and Prioritization Module has taught me about the processes of development of APIs, database connectivity, as well as communication between services in a distributed environment. Additionally, I got a chance to learn about the significance of teamwork in project development.

In general, the project was able to effectively show how current software architectures can be used to make disaster relief more coordinated. Further enhancements might involve the use of real-time analysis, AI-powered incident prioritization, monitoring tools, and containerization, through the use of Docker and Kubernetes.